

To display the contents of a file as it was at a particular commit, you can run the `git show <hash>:<filename>` command. But GUI Git tools will provide a more convenient interface for examining the repo log than the command line Git tool provides.

Recovering Old Changes

Let's say you want to work with an earlier version of your source code because you've introduced a bug, or perhaps you accidentally deleted a file. A version control system lets you undo, or *roll back*, your working copy to the content of an earlier commit. The exact command you'll use depends on the state of the files in the working copy.

Keep in mind that version control systems only add information. Even when you delete a file from a repo, Git will remember it so you can restore it later. Rolling back a change actually adds a new change that sets a file's content to its state in a previous commit. You'll find detailed information on various kinds of rollbacks at <https://github.blog/2015-06-08-how-to-undo-almost-anything-with-git/>.

Undoing Uncommitted Local Changes

If you've made uncommitted changes to a file but want to revert it to the version in the latest commit, you can run `git restore <filename>`. In the following example, we modify the *README.md* file but don't yet stage or commit it:

```
C:\Users\Al\wizcoin>git status
On branch master

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
C:\Users\Al\wizcoin>git restore README.md
C:\Users\Al\wizcoin>git status
On branch master
Your branch is up to date with 'origin/master'.
```